← Milestones • Active Sprint

Week 2: Learning & Discussion: MFS Dual-Portal Architecture Overview

Study architecture differences between customer-facing `iBanking_portal` and back-office `adminconsole`. Understand authentication flows (Customer OTP/JWT vs. Admin RBAC & session management), API gateway routing, and Serverless service communication.

WEEK 2 SPRINT OUTLINE Oct 12 - Oct 16, 2026

Learning & Discussion: MFS Dual-Portal Architecture Overview

Study architecture differences between customer-facing `iBanking_portal` and back-office `adminconsole`. Understand authentication flows (Customer OTP/JWT vs. Admin RBAC & session management), API gateway routing, and Serverless service communication.

WEEK 2 SPRINT PILLARS & OBJECTIVES

Pillar 1 — Portal Architecture
Understand the differences between `iBanking_portal` and `adminconsole`.

Pillar 2 — Customer Authentication
Study OTP verification, JWT access tokens and refresh tokens.

Pillar 3 — Admin Authentication
Understand RBAC, roles, permissions and session management.

Pillar 4 — API Gateway
Learn API routing, authentication policies and request handling.

Pillar 5 — Serverless Services
Understand communication between Azure Functions and backend services.

Pillar 6 — Security & Testing
Review authorization, error handling, logging and audit trails.

Pillar 7 — Documentation
Prepare an architecture diagram and summarize key findings.

INTERNAL DEPENDENCIES — Dependencies Project Deep Dive
Rationale, usage points, and runtime execution of internal shared libraries.

Lead Mentor: Ko Hlwan Kyaw Phyo • **Interns:** Ma Yamin Aye & Mg Phyo Min Htet •

Deliverable: Weekly Report #2 + Architecture Diagram

`Source: milestones/week_2/plan.md`

Repositories: `iBanking_repo/ibanking-spa` `iBanking_repo/ibanking17` `adminConsole_repo/admin_console` `adminConsole_repo/admin_console_spa` • Plan Source: `milestones/week_2/plan.md`

Week 2 Day-by-Day Sprint Schedule

Synchronized with the MFS Journey Plan daily milestones (Oct 12 – Oct 16, 2026).

5 Days Sprint

DAY 1 • MON Oct 12

Portal Architecture

Compare iBanking_portal and admin_console_spa project directory structures, modules, and component trees.

DAY 2 • TUE Oct 13

Customer Auth (OTP)

Trace SMS gateway OTP dispatch, JWT claims, and Angular silent refresh token interceptors.

DAY 3 • WED Oct 14

Admin RBAC & Tokens

Inspect admin_user, roles, permissions, session idle timeout, and blacklisttoken table.

DAY 4 • THU Oct 15

Gateway & Serverless

Azure Functions host.json, local.settings.json, httputils, and sequelize-java code paths.

DAY 5 • FRI Oct 16

Audit & Deliverable

Assemble MFS Architecture diagram, verify schema dump, and submit Weekly Report #2.

Portal Architecture

1. Portal Architecture: Understand the differences between iBanking_portal and adminconsole

#1

Analyze the core differences in user personas, operational scope, project structure, and state management between retail banking and back-office administrative portals.

Dimension	iBanking Portal (Customer)	Admin Console (Back-Office)
Target Audience	Retail & SME Bank Customers	Bank Operations, Risk Officers, System Admins
Primary Tech	Angular 17+ Customer SPA (ibanking-spa / ibanking17)	Angular Back-Office SPA (admin_console_spa)
Security Stance	Self-service, OTP, Session timeout	Strict RBAC, Maker-Checker, Immutable Audit Log
Data Operations	Account balances, fund transfers, bill pay	User management, limit overrides, audit trail inspection

REQUIRED ACTION ITEMS:

- [ ] Review Angular directory hierarchy in admin_console_spa vs ibanking-spa / ibanking17
- [ ] Compare component hierarchy, services, and routing models between both portals
- [ ] Analyze routing, component lazy loading, and state services in Admin Console

Customer Authentication

2. Customer Authentication: Study OTP verification, JWT access tokens and refresh tokens

#2

Examine end-to-end customer authentication: credential validation, SMS/Email OTP challenge, JWT access token issuance, and silent refresh token rotation.

1. Login Request
Customer submits credentials; backend validates against customer DB schema.

2. OTP Challenge
6-digit short-lived OTP generated, salted & stored with TTL; sent via SMS gateway.

3. JWT Issuance
On OTP match, server issues short-lived JWT Access Token (15m) and secure Refresh Token.

4. Token Rotation
Angular HTTP Interceptor refreshes Access Token automatically before expiry.

REQUIRED ACTION ITEMS:

- [ ] Trace OTP generation, validation, and expiry timeout handling in backend
- [ ] Inspect JWT payload claims (sub, exp, iss, customerId, permissions)
- [ ] Verify refresh token storage in secure HTTP-only cookies vs memory storage

Admin Authentication

3. Admin Authentication: Understand RBAC, roles, permissions and session management

#3

Examine how back-office administrative users are authenticated and governed using granular Role-Based Access Control (RBAC).

Super Admin RBAC
Full tenant administration, system settings, role provisioning

Checker / Approver RBAC
Review and approve/reject maker transactions and customer status overrides

Maker / Operator RBAC
Initiate administrative adjustments, initiate customer unlocks

Auditor / Compliance RBAC
Read-only access to immutable audit trails, system access logs

---

REQUIRED ACTION ITEMS:

- [ ] Inspect Role, Permission, and AdminUser tables in PostgreSQL schema
- [ ] Trace API authorization filters checking required permissions on incoming requests
- [ ] Examine admin session idle timeout mechanisms (10-15 minute threshold)

API Gateway

4. API Gateway: Learn API routing, authentication policies and request handling

#4

Study how frontend requests route to backend microservices, how authentication headers are inspected, and how downstream service endpoints are organized.

REQUIRED ACTION ITEMS:

- [ ] Review API endpoint URL patterns routing /api/admin/* and /api/customer/*
- [ ] Trace how client HTTP interceptors attach Bearer tokens and correlation headers
- [ ] Understand downstream service request/response flow between frontend SPAs and backend APIs

Serverless Services

5. Serverless Services: Understand communication between Azure Functions and backend services

#5

Understand serverless execution in the admin_console backend, reviewing host.json, local.settings.json, and Azure Functions runtime triggers.

`host.json`

Specifies runtime logging levels, extension bundles, and function timeout policies.

`local.settings.json`

Local environment secrets, DB connection strings, and Azure storage connection.

`pom.xml`

Includes azure-functions-java-library and azure-functions-maven-plugin packaging.

REQUIRED ACTION ITEMS:

- [ ] Inspect host.json and local.settings.json in admin_console
- [ ] Run Azure Functions locally using 'mvn azure-functions:run'
- [ ] Examine communication channels between HTTP-triggered functions and backend DB

Security & Testing

6. Security & Testing: Review authorization, error handling, logging and audit trails

#6

Audit trails are non-negotiable in financial systems. Learn how audit logs record 'Who, What, When, IP, Before/After State' for every mutation.

REQUIRED ACTION ITEMS:

- [ ] Review audit log schema in PostgreSQL (cleandb.sql / report.sql)
- [ ] Verify centralized error handling preventing stack trace leakage to clients
- [ ] Analyze sensitive data masking (sanitizing passwords, tokens, and PII from log output)

Documentation

7. Documentation: Prepare an architecture diagram and summarize key findings

#7

Produce technical architecture artifacts, sequence diagrams, and key findings to present to mentors and engineering leadership.

REQUIRED ACTION ITEMS:

- [ ] Draft system architecture diagram illustrating client SPAs, Gateway, Functions, and DB
- [ ] Document key security differences between customer portal and admin console
- [ ] Compile Week 2 findings into executive report for manager review

INTERNAL DEPENDENCIES

Dependencies Project Deep Dive

Rationale, usage points, and runtime execution of internal shared libraries.

HTTP Client

Java 1.8 / 17

httputils

`com.starless.http:httputils`

WHY IS IT NEEDED?

Provides a unified, robust communication client across distributed services. Instead of each microservice writing repetitive boilerplate for HttpClient, SSL truststores, timeouts, retries, and header propagation, httputils standardizes error handling, correlation IDs, and JSON serialization.

WHERE IS IT USED?

Used across backend services (such as Azure Functions in admin_console and ibanking17) whenever an outbound HTTP request is needed to reach payment gateways, core banking APIs, or authentication servers.

HOW IT WORKS BEHIND THE SCENES

Wraps Jakarta RESTful Web Services (JAX-RS) and Jackson Databind. It applies interceptors for request/response logging, injects standard security/authorization headers, automatically maps HTTP error codes to strongly-typed banking exceptions, and unmarshals JSON payloads directly into Java DTOs.

Custom ORM

Postgres / MSSQL

sequelize-java

`com.starless:sequelize-java`

WHY IS IT NEEDED?

Provides a lightweight, intuitive Sequelize-style query builder pattern without the heavy memory footprint, reflection overhead, and complex lifecycle caching of Hibernate/Spring Data JPA in serverless environments.

WHERE IS IT USED?

Used extensively in backend data access layers and Azure Functions to perform CRUD operations, joins, and transactional mutations against PostgreSQL and MSSQL databases.

HOW IT WORKS BEHIND THE SCENES

Translates fluent method chains (such as findOne, findAll, create, update, and transaction blocks) into parameterized SQL at runtime. It secures against SQL injection through prepared statements, handles database connection pooling efficiently, and maps SQL result sets to Java entity objects using Lombok accessors.